

RESEARCH

Open Access



A simpler approach to inter-application authentication utilising cloud-native identities

Adrian Asher^{1*} and Raj Rajarajan¹

Abstract

Application Programming Interfaces (APIs) are widely consumed by both internal and external entities. When services authenticate to other services, they typically rely on static credentials such as API keys, username-password pairs, or mutual Transport Layer Security (mTLS) certificates. These static credentials have contributed to several major security breaches due to inadvertent or malicious disclosure. This paper proposes a cloud-agnostic authentication framework that replaces static credentials with cloud-native bootstrap identities. The approach works by having a calling application cryptographically prove its entitlement to a cloud-assigned role (such as an AWS IAM Role, GCP Service Account, or Azure Managed Identity) to an organisational identity provider (Org IDP), which validates the proof against the cloud platform and issues a short-lived JSON Web Token (JWT) conforming to the OpenID Connect (OIDC) standard. Receiving applications authenticate the caller using standard OIDC validation, requiring no shared secrets. The framework is evaluated through a proof of concept implementation on AWS, with the identity proof mechanisms for GCP and Azure independently validated on their respective platforms. Performance benchmarks demonstrate that JWT validation adds only 1–2 ms of overhead per request, and a threat analysis shows that the model removes reliance on static credentials, thereby eliminating a major class of secret-leakage risk whilst reducing man-in-the-middle and replay attack surfaces.

Keywords Cloud-native security, Workload identity, Credential-free authentication, Identity federation, OIDC

Introduction

As organisations increasingly migrate to the cloud, many Application Programming Interfaces (APIs) are either being migrated from on-premises environments or developed natively within cloud platforms. Despite this shift, the methods used to secure inter-service communication continue to rely on legacy authentication models, particularly static credentials such as API keys. These approaches originated in on-premises datacenter

environments where options for authentication and identity were limited.

Limitations of static credentials

Reliance on static credentials between applications has led to significant security breaches when credentials were inadvertently or maliciously exposed. In the Toyota breach of 2022, a secret key was accidentally committed to a public GitHub repository and remained exposed for five years, potentially allowing unauthorised access to customer data [1]. InfoSys in 2022 leaked full administrative AWS access keys on the Python Package Index (PyPI) for over a year [2]. In 2019, Equifax exposed API credentials through a publicly accessible server, enabling unauthorised access to sensitive systems [3]. Uber's 2014 breach similarly stemmed from AWS secret keys

*Correspondence:

Adrian Asher
adrian.asher@citystgeorges.ac.uk

¹Institute for Cyber Security, City St George's, University of London, London, UK

committed to a GitHub repository, granting attackers access to rider and driver data [4].

The scale of this problem is substantial and worsening. GitGuardian's 2024 State of Secrets Sprawl report [5] found 12.8 million secret occurrences on public GitHub repositories in 2023, a 28% increase from the prior year, with 91.6% of leaked valid secrets remaining active five days after detection. The 2025 report [6] found this had grown to 23.8 million new hardcoded secrets in 2024, a further 25% surge, with 35% of private repositories also containing hardcoded secrets. Verizon's 2024 Data Breach Investigations Report [7] found that stolen credentials were involved in 31% of all breaches over the preceding decade, and IBM's Cost of a Data Breach report [8] found that breaches involving compromised credentials take an average of 292 days to identify and remediate – longer than any other attack vector. In December 2024, Chinese state-sponsored attackers breached the U.S. Treasury Department by exploiting a single compromised API key belonging to a third-party provider [6]. In OWASP's API Security Top 10, broken authorisation is ranked first and broken authentication second [9], further demonstrating how prevalent authentication failures remain in APIs today.

Why existing solutions are insufficient

Several existing approaches attempt to address inter-application authentication, but each carries limitations that leave the core problem unresolved.

OAuth 2.0 client credentials [10]

The OAuth 2.0 client credentials grant is widely used for service-to-service authentication. However, it still requires a client secret to be provisioned, stored, and rotated – the same static credential lifecycle that creates leakage risk. The client secret itself becomes the attack surface.

Mutual TLS (mTLS)

Certificate-based authentication through mTLS eliminates shared secrets but introduces significant operational complexity. A private certificate authority (CA) must be provisioned and managed, certificates must be rotated frequently, and expired certificates cause outages. Service meshes such as Istio [11] and Linkerd [12] automate mTLS between services within a single cluster but do not address cross-cluster or cross-cloud authentication, and require adopting the mesh infrastructure itself.

Zero trust architectures

Frameworks such as NIST SP 800–207 [13] advocate for continuous verification of every request. However, zero trust is a set of principles rather than an authentication mechanism; it does not prescribe how services

should prove their identity to one another. The proposed model in this paper can serve as an implementation of the identity verification component within a zero trust architecture.

SPIFFE/SPIRE [14]

The Secure Production Identity Framework for Everyone (SPIFFE) provides a standard for workload identity through SPIFFE Verifiable Identity Documents (SVIDs). SPIRE, its reference implementation, issues X.509 or JWT SVIDs to workloads. While SPIFFE addresses workload identity, it requires deploying and operating the SPIRE infrastructure, including attestation agents on every node. The model proposed in this paper achieves a similar outcome – workload identity without static credentials – by leveraging the identity infrastructure already provided by the cloud platform, requiring no additional agents or infrastructure beyond the organisational IDP.

Cloud-native single-provider mechanisms

Individual cloud providers offer mechanisms for their own ecosystems. AWS IAM Roles Anywhere [15] extends IAM roles to workloads outside AWS using X.509 certificates (inbound to AWS). AWS Outbound Identity Federation [16], announced at re:Invent 2025, allows AWS workloads to obtain OIDC tokens for authenticating to external services. GCP Workload Identity Federation [17] enables external workloads to access GCP resources. However, each of these addresses only one direction or one cloud provider. None provides a unified, cloud-agnostic framework for inter-application authentication across heterogeneous environments.

Motivation and scope

The limitations described above share a common thread: either they depend on static credentials that can leak, they require substantial new infrastructure, or they are confined to a single cloud provider's ecosystem. Cloud platforms already provision a bootstrap identity to every compute resource – AWS IAM Roles, GCP Service Accounts, Azure Managed Identities – for the purpose of accessing platform services. This paper proposes repurposing these existing bootstrap identities as the foundation for inter-application authentication, removing the need for static credentials.

The proposed approach is not specific to any single cloud provider. While the proof of concept implementation uses AWS, the architectural model is demonstrated for AWS, GCP, and Azure, and the approach generalises to any cloud platform that provisions compute with a bootstrap identity.

In doing so, the paper addresses two key questions:

- What security benefits do cloud-native identities offer compared to static credentials?
- How does the performance of a cloud-native identity model compare with traditional methods like API keys?

The main contributions of the paper are as follows:

- A cloud-agnostic architectural pattern for application-to-application authentication that removes reliance on static credentials by leveraging cloud-native bootstrap identities through an organisational IDP;
- A proof of concept implementation on AWS demonstrating the end-to-end authentication flow, with empirical validation of the identity proof mechanisms on GCP and Azure;
- A performance evaluation showing that JWT-based validation adds minimal latency overhead compared to static credential approaches;
- A threat analysis comparing the security posture of the proposed model against static credential methods across multiple attack vectors;
- A comparison with the subsequently released AWS IAM Outbound Identity Federation, demonstrating the proposed model's differentiation for multi-cloud environments.

The remainder of this paper is organised as follows. Section “[Background and related work](#)” reviews background and related work. Section “[Proposed architecture and implementation](#)” describes the proposed architecture, its implementation across cloud providers, and the proof of concept environment. Section “[Results and evaluation](#)” presents the evaluation results including performance, fault tolerance, and threat analysis. Section “[Security analysis](#)” provides a consolidated security analysis covering threats addressed, vulnerabilities introduced, and custom JWT claims. Section “[Comparison with AWS outbound identity federation](#)” compares the proposed model with AWS Outbound Identity Federation. Section “[Conclusions and future work](#)” concludes and identifies future work.

Background and related work

The evolution of application architecture

In modern software design, micro-service architectures have become prevalent. Applications are no longer monolithic in composition; instead, they comprise a number of smaller, interconnected services, each with its own compute resources, data store, and interactions with other services.

For these services to interact securely, each must validate the source of inbound API calls. In the absence of

user-based authentication mechanisms such as passwords, it became necessary to authenticate the API calls between the services themselves.

Current application to application authentication approaches

Applications typically authenticate to other applications using one of the following methods:

- API key
- Username and password pair
- Certificate and key pair

These are collectively referred to as static credentials, which may be rotated periodically. Possession of such credentials grants the ability to invoke the target API; consequently, they must be stored securely, typically within a secrets vault. Access to the vault is itself authenticated through a cloud-native bootstrap identity via role-based authorisation.

Each credential must be managed throughout its life-cycle, including secure storage, regular rotation, and synchronisation between the client and target service.

The credentials are used to invoke APIs synchronously, with REST APIs being a common use case. The first two methods require the credential to be known by both the calling client and the receiving service [18].

Certificate and key pair authentication operates differently: the client produces a proof of possession and presents it to the receiving service, commonly as part of mutual Transport Layer Security (mTLS) authentication.

Related work

In this section, the related research is reviewed and contrasted with the approach proposed in this paper.

Secret detection in source Code

A significant body of research has focused on detecting leaked credentials after they have been committed to code repositories. Sinha et al. [19] proposed methods for version control systems to detect credentials upon check-in. Meli et al. [20] conducted a large-scale characterisation of secret leakage in public GitHub repositories, finding thousands of exposed credentials daily. Krause et al. [21] studied prevention and remediation strategies against accidental secret leakage, and Lykousas and Patsakis [22] developed automated detection techniques whilst assessing the strength of developers' credential choices. These approaches are reactive – they detect credentials after exposure. The model proposed in this paper takes a fundamentally different approach by removing reliance on static credentials, thereby eliminating the primary source of leakage risk.

Intra-cloud and inter-cloud Authentication

Walsh and Manferdelli [23] investigated authentication challenges between services within and across cloud environments, proposing Cloudproxy TLS for establishing trust between cloud-hosted applications. Their work highlights the complexity of cross-cloud trust but relies on certificate-based mechanisms with their associated management overhead. The model proposed in this paper achieves cross-cloud authentication through a simpler token exchange pattern using existing cloud bootstrap identities.

Behavioural approaches

In [24], the authors propose a privacy-preserving approach to application-to-application authentication using dynamic runtime behaviours, attempting to determine whether a credential is being presented by an authorised entity based on how and in what manner the requesting server operates. This research focuses on protecting the credential rather than changing the authentication mechanism. In contrast, the model proposed in this paper removes the static credential entirely.

Token exchange standards

RFC 8693 [25] defines the OAuth 2.0 Token Exchange protocol, providing a framework for exchanging one security token for another. The model proposed in this paper can be understood as an instance of this pattern: a cloud-native proof of identity (such as a signed AWS STS request) is exchanged at the Org IDP for an OIDC JWT. However, RFC 8693 does not prescribe the use of cloud bootstrap identities as the source token, which is the principal contribution of this work.

Industry adoption of cloud-native workload identity

Subsequent to the development of the model proposed in this paper, AWS announced IAM Outbound Identity Federation at re:Invent 2025 [16]. This feature introduces a new STS API, `GetWebIdentityToken`, which allows AWS workloads to obtain a signed OIDC JWT directly from AWS STS, using their existing IAM role credentials. The JWT can then be presented to external services such as Azure, GCP or SaaS platforms for authentication without any static credentials. AWS cited that two-thirds of customer security incidents stemmed from unauthorised actors using valid long-term credentials [26], reinforcing the motivation underpinning the model proposed in this paper.

This development provides significant industry validation of the core thesis of this paper: that cloud-native bootstrap identities can and should replace static credentials for inter-application authentication. However, there are important architectural distinctions between the AWS native feature and the model proposed here,

which are discussed in section “[Comparison with AWS outbound identity federation](#)”.

Proposed architecture and implementation

This section presents the proposed architectural model, describes how it is realised across AWS, GCP, and Azure, and details the proof of concept implementation environment.

Architectural overview

This paper proposes the use of cloud platform bootstrap identities as an alternative to static credentials for inter-application authentication.

A *bootstrap identity* is defined here as a platform-provisioned credential automatically assigned to a compute resource at instantiation, granting it the ability to authenticate to cloud platform services without any manual credential configuration. Examples include AWS IAM Instance Profiles (which provide temporary security credentials via the EC2 metadata service), GCP Service Accounts (attached to compute instances at creation), and Azure Managed Identities (accessible via the Instance Metadata Service). These identities are managed entirely by the cloud platform, including key generation, rotation, and revocation.

Since the bootstrap identity itself constitutes a credential, a mechanism is required to provide proof of possession without transmitting the credential directly – analogous to the approach used in mTLS. All major cloud platforms provision compute resources with a bootstrap identity, enabling access to platform services such as storage, message queues, and data stores. Within the AWS Identity and Access Management (IAM) system, one key component is the AWS Security Token Service (STS). AWS STS provides several identity management functions, including the ability for a compute resource (a *principal*) to assume a role. The principal must be authorised to assume the role, and upon doing so, it can exercise the permissions associated with that role through ephemeral temporary security credentials.

A second function within AWS STS, `GetCallerIdentity`, enables a caller to obtain confirmation of its current identity and role. To invoke this function, the caller must cryptographically prove its identity to the STS service. In standard operations, a user or compute service prepares and executes this call directly. In the approach proposed in this paper, the client prepares the `GetCallerIdentity` request, including a digital signature, and passes the pre-signed request to a separate service for execution. When the second service executes the request, AWS STS returns the identity of the original signer.

For workloads in Microsoft Azure and Google Cloud Platform (GCP), different proof mechanisms are

required, as outlined in section “[Proving entitlement to roles across cloud providers](#)”. Many other cloud providers, including private cloud platforms, offer similar capabilities. In each case, the cloud provider provisions a bootstrap identity to the compute service as part of the underlying platform infrastructure.

Through these mechanisms, it can be verified that the service presenting the request was entitled to exercise the claimed role and identity, as it would not otherwise possess the credentials necessary to generate a valid cryptographic signature or access token.

To avoid requiring each receiving application to independently validate cloud-specific identity proofs, a trusted organisational identity provider (referred to as the Org IDP in this paper) is introduced as an intermediary.

The client application submits its proof of role entitlement to the Org IDP, which validates it against the cloud platform (e.g., via AWS STS `GetCallerIdentity`) and, upon confirmation, issues a signed JWT conforming to the OpenID Connect (OIDC) standard [27]. The JWT lifetime is configurable; short lifetimes (typically 5–15 minutes) are recommended to minimise replay exposure. Receiving applications validate the JWT using standard OIDC protocols.

Figure 1 illustrates the authentication flow. The client Application (A) generates a signed request proving its entitlement to Role (rA) and submits this to the Org IDP. The IDP validates the entitlement via AWS STS and, upon confirmation, generates and returns a signed JWT.

Application (A) then presents this JWT when invoking Application (B). Application (B) validates the JWT by retrieving the Org IDP’s public keys and verifying the signature, confirming that the caller is entitled to Role (rA). No API keys or shared secrets are exchanged between the two applications.

The Org IDP requires an asymmetric key pair for signing JWTs. The private key must be stored securely, either in a key vault or through a cloud-managed service such as AWS KMS or CloudHSM. The OIDC specification permits multiple concurrent public keys, facilitating key rotation without service interruption.

This architecture ensures that receiving applications implement standard OIDC authentication patterns, requiring no cloud-specific code. Client applications can obtain JWT claims through a provided SDK or library that performs the cloud-specific identity proof on their behalf, exchanging it for an Org IDP JWT without managing any static credentials.

Proving entitlement to roles across cloud providers

This section details how the proposed model utilises each cloud provider’s bootstrap identity mechanism for inter-application authentication across AWS, GCP, and Azure.

1) **AWS:** In AWS, the application must provide a cryptographically assured proof of its role entitlement to the Org IDP. This proof is constructed using the Access Key, Secret Key, and Session Token provisioned to the compute resource by the AWS Security Token Service (STS).

The pseudo code to produce this proof is as follows:

- Generate URL
<https://sts.eu-west-2.amazonaws.com>
`?Action=GetCallerIdentity&Version=2011-06-15`
- Sign Request (including headers) with the AWS IAM Role STS Session Token and temporary keys provided by the platform, using HMAC-SHA256.

Resulting in:

```
Authorization: AWS4-HMAC-SHA256
Credential=CRED/20230524/eu-west-2/sts/aws4_request
SignedHeaders=host;range;x-amz-date
Signature=fe5f896317...6630326f1024
```

Listing 1 HTTP headers

The generated signature is valid for 15 minutes, after which AWS STS will reject it. The application transmits the signed headers and request parameters to the Org IDP. Upon receipt, the IDP validates that all headers, the URL, and parameters conform to a pre-configured allow list. The IDP then constructs an HTTP request using the provided headers, including the signature, and submits it to AWS STS. The STS response takes the following form:

```
<GetCallerIdentityResponse xmlns=
https://sts.amazonaws.com/doc/2011-06-15/>
  <GetCallerIdentityResult>
    <Arn>arn:aws:sts::1234567890:assumed-role
/app-service-role/i-0abc123def</Arn>
    <UserId>AROEXAMPLEID:i-0abc123def</UserId>
    <Account>1234567890</Account>
  </GetCallerIdentityResult>
  <ResponseMetadata>
    <RequestId>uuid</RequestId>
  </ResponseMetadata>
</GetCallerIdentityResponse>
```

Listing 2 AWS STS `GetCallerIdentity` response

As this response originates directly from AWS STS, the IDP can trust the asserted identity. The IDP has now confirmed that the caller is entitled to the identity returned by AWS STS, and can issue an Org IDP JWT claim based on this assertion.

2) **GCP:** Google Cloud Platform (GCP) does not provide a direct equivalent to the AWS `GetCallerIdentity` API, necessitating a different proof mechanism. Figure 2 illustrates how the proposed model utilises the

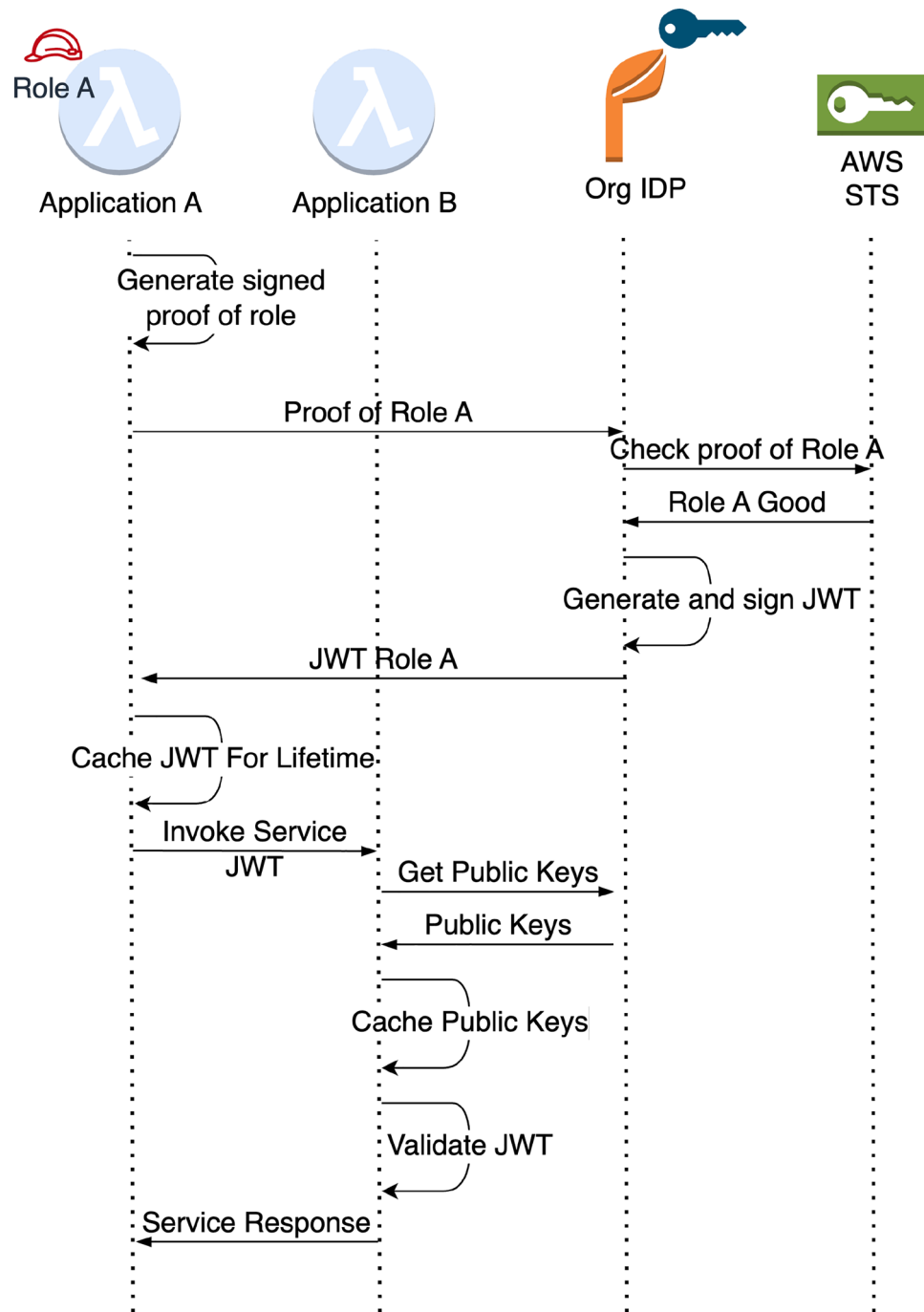


Fig. 1 Modern approach to app-to-app authentication in AWS

GCP Service Account API's `signBlob` capability to assert proof of an application's entitlement to a specific Service Account.

The GCP flow requires an additional round trip compared to the AWS approach. The Org IDP first generates a cryptographically random string (nonce) and returns it to the client application. The client then invokes the GCP `signBlob` API, which signs the nonce using the private key managed by GCP on behalf of the application's

Service Account. The client returns the signed nonce, its Service Account identifier, and the associated IAM OAuth scope to the Org IDP. The Org IDP retrieves the Service Account's public keys from the GCP Service Account metadata endpoint and validates the signature against the nonce it originally issued. The nonce serves a dual purpose: it prevents replay attacks (as each nonce is single-use and time-limited) and confirms that the caller possesses the ability to invoke `signBlob` on behalf of

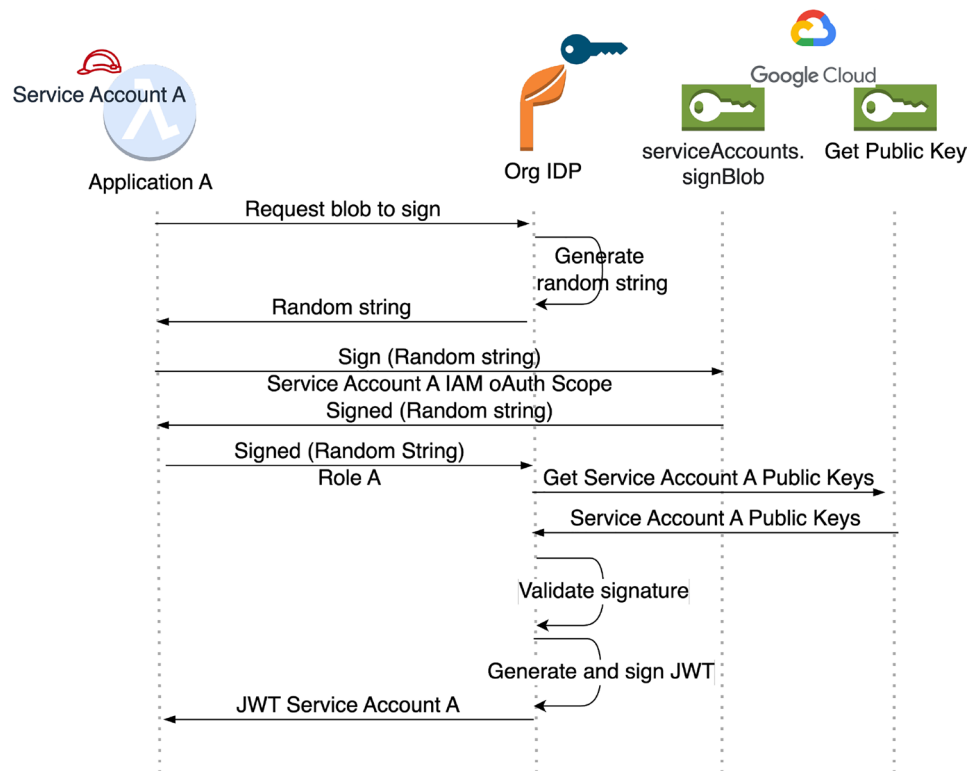


Fig. 2 A proposed implementation of a cloud identity based model for authentication between applications in GCP

the claimed Service Account. Upon successful validation, the Org IDP issues a JWT and the remainder of the authentication flow proceeds as described in section “[Architectural overview](#)”.

This flow was empirically validated on a GCP Compute Engine instance (e2-micro, europe-west2) with an attached Service Account. The `signBlob` round-trip completed in 80 ms, public key retrieval from the Service Account metadata endpoint in 82 ms, and RSA signature verification in approximately 5 ms. The tampered-nonce test confirmed that the signature is cryptographically bound to the exact challenge issued by the IDP.

3) *Azure*: Figure 3 illustrates the Azure implementation. Microsoft Azure provides a Managed Identity mechanism (the Azure equivalent of the bootstrap identity described earlier in this paper). This identity is accessible via the Azure Instance Metadata Service (IMDS) at the link-local address 169.254.169.254, available to any compute resource running in Azure, such as Azure Virtual Machines, Azure Container Instances, or Azure App Services. The application requests an access token from the IMDS endpoint, specifying the Org IDP’s application identifier as the audience. Azure Entra ID (formerly Azure Active Directory) issues a JWT scoped to the Org IDP. The application presents this JWT to the Org IDP, which validates the token’s signature against Azure Entra ID’s published public keys, confirms the audience claim matches its own identifier, and extracts the Managed

Identity’s object identifier as the caller’s identity. Upon successful validation, the Org IDP issues its own JWT and the remainder of the authentication flow proceeds as described in section “[Architectural overview](#)”. As the Azure proof is itself a time-limited JWT issued by Azure Entra ID, no additional nonce mechanism is required – the token’s expiry inherently constrains replay opportunities.

This flow was empirically validated on an Azure Virtual Machine (Standard_B1s, UK South) with a system-assigned Managed Identity. Token acquisition from IMDS completed in 286 ms, Entra ID public key retrieval in 354 ms, and RSA signature verification in under 1 ms. The verified claims confirmed the Managed Identity’s object identifier and tenant, demonstrating that a third party can independently validate the Azure bootstrap identity using only publicly available Entra ID keys.

Cross-cloud authentication

A key advantage of the Org IDP model is that it enables authentication between applications running on different cloud providers without either application needing awareness of the other’s cloud platform. Because the Org IDP accepts proof of identity from AWS, GCP, and Azure, and issues a unified OIDC JWT regardless of the originating cloud, the receiving application validates the same token format irrespective of where the caller is hosted.

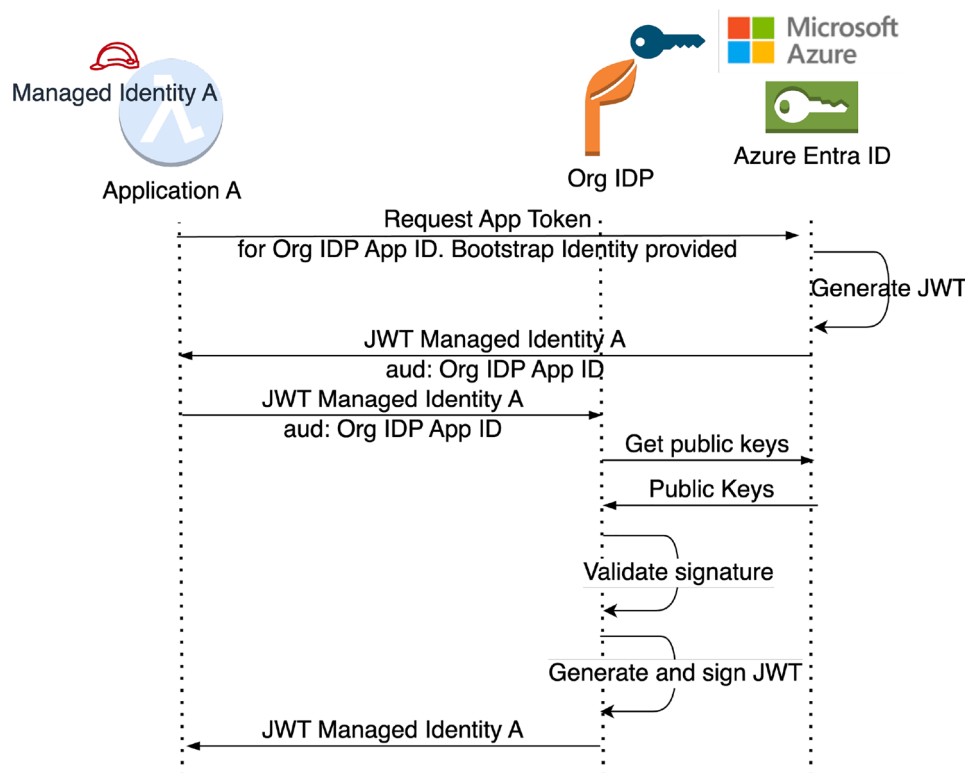


Fig. 3 A proposed implementation of a cloud identity based model for authentication between applications in Azure

Consider a concrete scenario: an application running on AWS needs to call an application running on GCP. With the proposed model, the AWS application generates a signed `GetCallerIdentity` request proving its IAM Role, submits it to the Org IDP, and receives a JWT. The GCP application validates this JWT using standard OIDC mechanisms – retrieving the Org IDP’s public keys and verifying the signature. At no point does the GCP application need to understand AWS IAM, STS, or Signature Version 4. The same flow operates in reverse: a GCP application proves its Service Account identity to the Org IDP via `signBlob`, receives a JWT, and an AWS application validates it identically.

This cross-cloud pattern eliminates the need for cloud-specific federation configurations between each pair of providers. Without the Org IDP, an organisation requiring bidirectional authentication across three clouds would need to configure up to six bilateral federation relationships (AWS-to-GCP, GCP-to-AWS, AWS-to-Azure, Azure-to-AWS, GCP-to-Azure, Azure-to-GCP), each using provider-specific mechanisms. Even organisations requiring only a subset of these directions face configuration complexity that grows quadratically with the number of providers. The Org IDP reduces this to three inbound integrations (one per cloud provider) and a single outbound JWT format, simplifying both the

configuration and the receiving application’s authentication logic.

This cross-cloud capability is not addressed by any single cloud provider’s native identity features. AWS Outbound Identity Federation issues AWS-specific tokens that GCP or Azure must be individually configured to accept. The proposed model abstracts this complexity behind a single organisational trust boundary.

Proof of concept implementation

The proof of concept was implemented on AWS to validate the proposed architectural model end-to-end. The implementation comprised three components:

Client application

A Java application deployed on an Amazon EC2 instance (t3.medium, eu-west-2 region) with an IAM instance profile providing the bootstrap identity. The application used the AWS SDK for Java (v2) to generate signed `GetCallerIdentity` requests using AWS Signature Version 4 [28]. The signed request headers and URL were forwarded to the Org IDP via HTTPS.

Organisational IDP

A Java application deployed as an AWS Lambda function. Upon receiving a signed request from a client, the IDP validated the headers and parameters against a

pre-configured allow list, forwarded the signed request to AWS STS, and upon successful identity confirmation, issued a signed JWT. JWT signing used RSA-256 keys managed through AWS KMS. The IDP exposed a standard OIDC discovery endpoint (`.well-known/openid-configuration`) and a JSON Web Key Set (JWKS) endpoint for public key distribution.

Receiving application

A Java application deployed on a separate EC2 instance that validated inbound JWTs using standard OIDC libraries. The application retrieved the Org IDP's public keys via the JWKS endpoint and cached them in accordance with the HTTP cache-control headers.

The JWT libraries used for validation benchmarking were `io.jsonwebtoken` (v0.12.3) and `nimbus-jose-jwt` (v9.37), both running on OpenJDK 17. Performance tests were executed on a 2.5 GHz Intel Xeon W core, single-threaded, with 10,000 iterations per measurement.

Results and evaluation

Performance evaluation

To evaluate the performance characteristics of the proposed model, a series of benchmarks were conducted measuring each stage of JWT validation. The steps required to validate a JWT claim are as follows:

- Retrieve IDP public keys
- Split JWT into header, payload, signature
- Cryptographically validate signature against public keys

Public key retrieval can be cached and typically completes within 100 ms. In testing, retrieval from a co-located region completed within 30 ms; cross-region retrieval remained below 100 ms when the regions were within the same continent.

Signature validation exhibited more variation. Testing with two libraries revealed substantial differences in both initialisation time and first-validation latency, likely attributable to deferred object instantiation and cache population during the first validation. After initialisation, subsequent validations completed in approximately 1–2 ms depending on the library.

Each measurement was averaged over 10,000 iterations executed on a single thread on a 2.5 GHz Intel Xeon W core. Memory utilisation was negligible, as the cryptographic operations were CPU-bound. HMAC-SHA256

was used as the signing algorithm for these benchmarks. It should be noted that HMAC-SHA256 is a symmetric algorithm requiring a shared secret between the Org IDP and each receiving application; in a production deployment, the shared key must be securely distributed, rotated, and versioned across all parties. An asymmetric algorithm such as RS256 (as used in the proof of concept Org IDP via AWS KMS) avoids this shared-secret management by allowing receivers to validate using only the IDP's public key. Asymmetric verification is slower: the GCP and Azure validation tests measured RSA signature verification at approximately 5 ms and under 1 ms respectively. The "Average" column in Table 1 reflects HMAC-based validation of a previously unseen JWT against cached keys.

By comparison, validating a static API key typically involves a string comparison or hash lookup, which completes in microseconds. The proposed model therefore introduces an overhead of approximately 1–2 ms per request for JWT validation on the receiving side. For the majority of API calls, where network round-trip latency is in the tens of milliseconds, this overhead is negligible. For latency-sensitive applications, caching validated claims for their remaining lifetime can reduce subsequent validations to a performance level comparable with static credentials.

It should be noted that the benchmarks presented here measure only the JWT validation step at the receiving application. The full end-to-end token acquisition flow – comprising the client's generation of a signed request, the Org IDP's validation against the cloud platform (e.g., the round-trip to AWS STS), JWT generation and signing, and the return to the client – incurs additional latency. However, this cost is amortised over the token's lifetime: once a JWT is obtained, it is reused for all subsequent API calls until expiry. In the proof of concept, token acquisition was observed to complete within 200–400 ms in the same AWS region, a cost incurred only at application startup or token renewal.

Fault-tolerance comparison

Any inter-application authentication solution must have availability at least equal to that of the applications it serves; otherwise, the authentication system itself becomes a single point of failure. Depending on the architecture, the solution may only need to be available at specific points in the application lifecycle, such as during startup or token renewal.

Inter-application authentication typically does not involve persistent connection pooling, but may utilise session tokens (such as OAuth claims). The authentication service must therefore be available when such tokens are created or refreshed.

Table 1 JWT validation times (10,000 iterations, single-threaded, 2.5 GHz Intel Xeon W, OpenJDK 17, HMAC-SHA256)

Library	Setup	1st	Average
<code>io.jsonwebtoken</code> (0.12.3)	130 ms	26 ms	2.01 ms
<code>nimbus-jose-jwt</code> (9.37)	13 ms	344 ms	1.59 ms

With legacy approaches, static credentials are stored in a cloud-based vault whose availability dictates whether new compute instances can start and retrieve their credentials. Similarly, the proposed model depends on the availability of the Org IDP for new instances to obtain their initial JWT.

In both approaches, caching mitigates the impact of temporary unavailability. Running applications can retain their credentials or tokens and continue operating while the vault or Org IDP is unavailable. However, in the proposed model, cached tokens have a finite lifetime; if a token's lifetime is set to two hours, the Org IDP must be restored within two hours of the oldest issued token to prevent authentication failures.

Therefore, both the legacy and proposed models are susceptible to outages caused by unavailability of their respective central services. The proposed model's token lifetime constraint can be mitigated by deploying a highly available, multi-region Org IDP architecture.

Credential lifecycle comparison

Even when a static credential is not compromised and is held securely, there remains a significant operational overhead in administering it. The lifecycle of a typical static credential comprises the following stages:

- (1) Creation
- (2) Storage in vault
- (3) Retrieval by application from the vault
- (4) Rotation by the vault to the new value
- (5) Deletion

At every stage, specific security requirements must be met: generation must ensure sufficient complexity and randomness; storage must be automated through the CI/CD pipeline without human knowledge of the credential; retrieval must be restricted to authorised entities; rotation must be performed atomically to avoid race conditions; and deletion must occur when the credential is no longer required.

Application outages frequently occur during the rotation stage, as the credential must be updated atomically across both the client and the service to avoid race conditions. In practice, rotation often requires coordinated downtime for both parties.

In contrast, the proposed model removes reliance on static credentials; consequently, there is no credential to rotate, and a major limitation of the legacy approach is overcome.

Threat vector analysis

Static credentials are vulnerable to a number of threat vectors, namely:

- *Compromised credential*: The credential may be committed to source code, stored in inappropriate locations, leaked through side channels, or exfiltrated from a compromised secrets manager.
- *Weak credentials*: API credentials may lack account lockout protections, rendering them susceptible to brute-force attacks if insufficiently complex.
- *Man-in-the-middle attacks*: Traffic may be intercepted by an untrusted party on a compromised network, though the prevalence of TLS has reduced this risk.
- *Replay attacks*: A captured authentication request could be replayed to establish an unauthorised session.

Each of these vectors was evaluated against the proposed model to determine whether the associated risk is removed, reduced, unchanged, or newly introduced relative to the static credential approach. The results are summarised in Table 2.

Compromised credentials and weak credentials

The proposed model removes reliance on static credentials, eliminating a major class of secret-leakage risk. However, if an attacker compromises the workload itself (e.g., through a remote code execution vulnerability), they may be able to use the workload's bootstrap identity to obtain valid tokens from the Org IDP. The model protects against static secret sprawl but does not address all consequences of full workload compromise.

Man-in-the-middle

In the event of a successful interception, captured JWT claims are valid only for their configured lifetime, unlike a static API key which remains valid indefinitely until revoked.

Replay attacks

The mitigation varies by cloud platform. In AWS, the signed `GetCallerIdentity` request includes a timestamp and is valid for only 15 minutes, limiting the replay window. In GCP, the proposed design incorporates a single-use nonce which the Org IDP stores and validates against reuse, providing the strongest replay protection. In Azure, the bootstrap identity proof is itself a

Table 2 Threat vector delta

Vector	Proposed Model
<i>Compromised Credentials</i>	Materially Reduced
<i>Weak Credentials</i>	Materially Reduced
<i>Man in the middle</i>	Risk Reduced
<i>Replay attacks</i>	Risk Reduced
<i>Weak ciphers</i>	Risk Introduced

time-limited JWT, constraining replay opportunities. It should be noted that token expiry reduces but does not fully prevent replay: during the validity window, a captured proof or JWT could still be replayed. Receiving applications should implement additional controls where required, such as binding tokens to source IP addresses or requiring unique transaction identifiers.

Weak ciphers

The proposed model introduces a dependency on the cryptographic strength of the signing algorithm used by the Org IDP. If weak ciphers are employed, an attacker could potentially forge claims. This risk is mitigated by enforcing minimum cipher strength requirements in both the Org IDP configuration and the receiving applications' validation logic.

Security analysis

This section consolidates the security implications of the proposed model, including the threat categories materially reduced by removing reliance on static credentials, the new attack surface introduced by the cloud identity approach, and the operational overhead removed.

Threats and operational overhead eliminated

The adoption of the proposed cloud-native authentication pattern materially reduces the following threat categories and removes the operational tasks that are otherwise inherent to static credential and mTLS-based approaches:

Threat categories addressed:

- *Credential leakage:* With no static credentials to manage, the risk of accidental or deliberate commitment of secrets to source code is removed for inter-application authentication.
- *Certificate expiry outages:* mTLS certificates that are not renewed cause service outages; the proposed model does not use certificates for inter-application authentication.

Operational overhead removed:

- *Credential creation and vaulting:* No static credentials need to be generated or stored.
- *Credential rotation:* No rotation is required, eliminating the race conditions and coordinated downtime associated with rotation (as discussed in Section "[Credential lifecycle comparison](#)").
- *Private CA management:* No private certificate authority is required for mTLS.
- *Certificate renewal:* No certificates require periodic rotation.

- *Secret scanning:* No credentials exist to be detected in source code, removing the need for scanning tools and false positive management.
- *Credential leakage response:* No incident response process is required for leaked credentials, including vendor notifications, threat intelligence correlation, and forensic log review.

Vulnerabilities introduced by the cloud identity approach

The use of cloud bootstrap identities introduces its own attack surface, which must be carefully managed. A notable precedent is a vulnerability discovered in HashiCorp Vault in 2020 [29], where the AWS authentication mechanism was exploited due to insufficient validation of the signed request parameters. The mitigation involved enforcing an allow list on both the headers and parameters of the signed request – a control that the proposed Org IDP design incorporates.

Each signed AWS request includes a timestamp, and the signature is valid for only 15 minutes, constraining the window for replay attacks. JWT claims issued by the Org IDP are similarly time-limited; the claim lifetime should be kept as short as operationally feasible to minimise exposure.

Workload compromise

It is important to state the boundary of the proposed model's threat coverage. If an attacker achieves full compromise of a workload (e.g., through remote code execution), they can access the workload's bootstrap identity and obtain valid tokens from the Org IDP. The proposed model is designed to protect against static secret sprawl and credential leakage – not against all consequences of workload compromise. Standard workload hardening practices remain essential, including restricting access to the cloud metadata service (IMDS). On AWS, IMDSv2 should be enforced to require session-based token retrieval; on Azure, network security groups should restrict IMDS access; and on GCP, metadata concealment should be enabled where supported.

Token revocation

Unlike static credentials, which can be revoked immediately in a vault, JWTs issued by the Org IDP are valid until their expiry time and cannot be individually revoked without maintaining a revocation list. This is an inherent trade-off of stateless token-based authentication. The primary mitigation is to keep token lifetimes short (e.g., minutes rather than hours). For scenarios requiring immediate revocation – such as a compromised workload – the Org IDP can be configured to stop issuing new tokens for the affected identity, and receiving applications can optionally implement a short-lived token denylist for the remaining validity period.

Custom JWT claims for authorisation

Any additional attributes that the Org IDP includes in the JWT are digitally signed by the IDP's keys. The JWT structure conforms to RFC 7519 [30], with standard claims including the issuer (*iss*), subject (*sub*), audience (*aud*), and expiration (*exp*).

Receiving applications must validate the following standard claims on every inbound JWT: *iss* must match the expected Org IDP issuer URL; *aud* must match the receiving application's identifier; *exp* must not have passed (with an acceptable clock skew tolerance, typically 30–60 seconds); and the signature must verify against the Org IDP's published JWKS keys. Failure to validate any of these fields would allow token misuse or cross-service attacks.

To enable receiving applications to make authorisation decisions about the caller's identity, the following custom claims are proposed:

- *region*: The cloud region in which the identity was generated.
- *crn*: The Cloud Resource Name – the cloud-specific identifier of the identity (e.g., an AWS ARN).
- *principal*: The role or service account the claim asserts.
- *principalType*: Whether the principal is a role, service account, or user.
- *accID*: The cloud account to which the principal belongs.

It is essential that the Org IDP restricts claim issuance to accounts within its trust boundary – for example, accounts belonging to the organisation's cloud infrastructure, with appropriate separation between production and non-production environments.

Environment separation can be further enforced through the audience claim: for instance, “city:prod” versus “city:dev”, ensuring that tokens issued for development environments cannot be used to authenticate against production services.

Comparison with AWS outbound identity federation

The release of AWS IAM Outbound Identity Federation [16] in November 2025 provides a useful point of comparison with the model proposed in this paper. Both approaches share the same fundamental principle: leveraging the cloud platform's bootstrap identity to produce short-lived OIDC tokens for inter-application authentication, thereby removing reliance on static credentials. Table 3 summarises the key differences.

Table 3 Comparison of proposed model with AWS Outbound identity federation

Aspect	Proposed Model	AWS Outbound
<i>Token Issuer</i>	Organisation IDP	AWS STS directly
<i>Cloud Scope</i>	AWS, GCP and Azure via unified IDP	AWS outbound only
<i>Cross-cloud</i>	Single IDP normalises identities across clouds	Requires separate federation per cloud
<i>Claim Control</i>	Organisation defines custom claims	AWS-defined claims under <code>sts.amazonaws.com</code> namespace
<i>Proof Mechanism</i>	Pre-signed <code>GetCallerIdentity</code> request	New <code>GetWebIdentityToken</code> API
<i>Infrastructure</i>	Requires deploying an Org IDP	No additional infrastructure

Architecture

The most significant distinction is the role of the intermediary. The proposed model introduces an organisational IDP that validates cloud bootstrap identities and issues its own JWT claims. AWS Outbound Identity Federation eliminates this intermediary by having STS itself act as the OIDC token issuer. The AWS approach is simpler for single-cloud AWS deployments as it requires no additional infrastructure. However, the Org IDP approach provides organisational control over claim contents, audience restrictions and trust boundaries that are independent of any single cloud provider.

Multi-cloud support

The proposed model's primary differentiator is its ability to federate identities from AWS, GCP and Azure through a single organisational IDP, producing a unified OIDC token format regardless of the originating cloud. AWS Outbound Identity Federation addresses only the outbound direction from AWS. An organisation operating across multiple clouds would still require separate federation mechanisms for GCP-to-Azure or Azure-to-GCP flows, a gap that the proposed Org IDP model addresses natively.

Organisational governance

The Org IDP model allows an organisation to enforce its own trust policies, such as restricting which cloud accounts or environments may receive tokens, defining custom claims for authorisation decisions, and maintaining a single audit point for all inter-application authentication. With the AWS native approach, these governance controls are distributed across individual IAM policies and cloud-provider-specific configurations.

In summary, the two approaches represent different points in a trade-off space. AWS Outbound Identity Federation offers a simpler deployment model with no additional infrastructure, making it well suited to AWS-only

environments. The proposed Org IDP model offers greater portability and centralised organisational control at the cost of deploying and operating an additional service. The choice between them depends on the organisation's multi-cloud requirements and governance needs.

Conclusions and future work

This paper addressed two key research questions:

What security benefits do cloud-native identities offer compared to static credentials?

The findings demonstrate that cloud-native identities materially reduce several categories of risk inherent to static credentials, including credential leakage and weak credential attacks. By replacing long-lived secrets with ephemeral, short-lived tokens bound to platform-attested identities, the attack surface is substantially reduced and the operational overhead of credential management is removed. The model does not address all consequences of workload compromise, which remains a separate concern requiring standard hardening practices.

How does the performance of a cloud-native identity model compare with traditional methods like API keys?

While static credentials provide near-instant authentication via simple string comparison, JWT validation introduces an overhead of approximately 1–2 ms per request. This overhead is negligible for the majority of production workloads and can be further reduced through token caching. The security and operational benefits of the proposed model outweigh this marginal latency cost.

In summary, the proposed architecture exchanges a cloud platform bootstrap identity for a short-lived OIDC token that can be used for inter-application authentication within an organisation. The approach generalises across cloud environments: by leveraging AWS STS, GCP Service Accounts, and Azure Managed Identities, applications can authenticate without reliance on static credentials.

The subsequent release of AWS IAM Outbound Identity Federation [16] provides independent industry validation of this approach, confirming that cloud providers recognise the necessity of moving beyond static credentials for workload authentication. The proposed model extends beyond any single provider's offering by providing a unified, multi-cloud identity federation pattern through an organisational IDP. Future work should investigate the standardisation of cross-cloud identity exchange protocols, the formal security analysis of the Org IDP trust model, and the performance characteristics of the proposed architecture under high-throughput production workloads.

Acknowledgements

Olga Limantseva, Ph.D.

Author contributions

A.A. performed the research, wrote the main manuscript text and prepared figures. All authors reviewed the manuscript.

Funding

The authors received no financial support for the research, authorship, and/or publication of this paper.

Data availability

No datasets were generated or analysed during the current study.

Declarations

Ethics approval and consent to participate

Not applicable.

Consent for publication

Granted.

Use of AI tools

The authors used AI-assisted tools (Anthropic) for LaTeX formatting and structural reorganisation of the manuscript. All research, experimental work, analysis, and intellectual contributions are solely the work of the authors, who take full responsibility for the content.

Competing interests

The authors declare no competing interests.

Received: 28 May 2025 / Accepted: 14 April 2026

Published online: 23 April 2026

References

1. GitGuardian (2022) Toyota breach. [Online]. Available: <https://blog.gitguardian.com/toyota-accidentally-exposed-a-secret-key-publicly-on-github-for-five-years/>. Last accessed 15 Oct 2023
2. Forbes T (2022) Infosys breach. [Online]. Available: <https://tomforbes.com/infosys-leaked-fulladminaccess-aws-keys-on-pypi-for-over-a-year/>. Last accessed 19 Oct 2023
3. H. VDP (2019) Equifax breach. [Online]. Available: <https://hackerone.com/reports/694931>. Last accessed 19 Oct 2023
4. Qz.com (2014) Uber breach. [Online]. Available: <https://qz.com/674520/companies-are-sharing-their-secret-access-codes-on-github-and-they-may-not-even-know-it>. Last accessed 15 Oct 2023
5. GitGuardian (2024) State of secrets sprawl report 2024. [Online]. Available: <https://www.gitguardian.com/state-of-secrets-sprawl-report-2024>. Last accessed 30 Mar 2026
6. GitGuardian (2025) The state of secrets sprawl 2025. [Online]. Available: <https://www.gitguardian.com/state-of-secrets-sprawl-report-2025>. Last accessed 30 Mar 2026
7. Verizon (2024) 2024 data breach investigations report. [Online]. Available: <https://www.verizon.com/business/resources/reports/dbir/>. Last accessed 30 Mar 2026
8. IBM Security (2024) Cost of a data breach report 2024. [Online]. Available: <https://www.ibm.com/reports/data-breach>. Last accessed 30 Mar 2026
9. <https://owasp.org/api-security/editions/2023/en/0xa2-broken-authentication/>, (2023). [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0xa2-broken-authentication/>
10. Hardt D (2012 Oct) The OAuth 2.0 authorization framework. RFC 6749. [Online]. Available: <https://www.rfc-editor.org/info/rfc6749>
11. Istio Authors (2024) Istio service mesh. [Online]. Available: <https://istio.io/latest/docs/concepts/security/>. Last accessed 30 Mar 2026
12. Linkerd Authors (2024) Linkerd automatic mTLS. [Online]. Available: <https://linkerd.io/2/features/automatic-mtls/>. Last accessed 30 Mar 2026
13. Rose S, Borchert O, Mitchell S, Connelly S (2020) Zero trust architecture. NIST special publication 800-207. [Online]. Available: <https://csrc.nist.gov/publications/detail/sp/800-207/final>

14. SPIFFE Authors (2024) SPIFFE: secure production identity framework for everyone. [Online]. Available: <https://spiffe.io/docs/latest/spiffe-about/overview/>. Last accessed 30 Mar 2026
15. Amazon Web Services (2022) IAM roles anywhere. [Online]. Available: <https://docs.aws.amazon.com/rolesanywhere/latest/userguide/introduction.html>. Last accessed 30 Mar 2026
16. Amazon Web Services (2025) Federating AWS identities to external services using IAM outbound identity federation. [Online]. Available: https://docs.aws.amazon.com/IAM/latest/UserGuide/id_roles_providers_outbound.html. Last accessed 30 Mar 2026
17. Google Cloud (2024) Workload identity federation. [Online]. Available: <https://cloud.google.com/iam/docs/workload-identity-federation>. Last accessed 30 Mar 2026
18. Farrell S (2009) Api keys to the kingdom. *IEEE Internet Comput* 13(5):91–93. <https://doi.org/10.1109/MIC.2009.100>
19. Sinha VS, Saha D, Dhoolia P, Padhye R, Mani S (2015) Detecting and mitigating secret-key leaks in source code repositories. In 2015 IEEE/ACM 12th Working Conference on Mining Software Repositories, pp 396–400
20. Meli M, McNiece MR, Reaves B (2019) How bad can it get? characterizing secret leakage in public github repositories. In NDSS
21. Krause A, Klemmer JH, Huaman N, Wermke D, Acar Y, Fahl S (2023) Poster: pushed by accident—prevention and remediation strategies against secret leakage. In In 19th Symposium on Usable Privacy and Security
22. Lykousas N, Patsakis C (2023) Tales from the git: automating the detection of secrets on code and assessing developers' passwords choices. In 2023 IEEE European Symposium on Security and Privacy Workshops (EuroSI&PW), pp 68–75
23. Walsh K, Manfredelli J (2017) Intra-cloud and inter-cloud authentication. In 2017 IEEE 10th International Conference on Cloud Computing (CLOUD), pp 318–325
24. Christodorescu M, Shirvanian M, Zawoad S (2022) Privacy-preserving application-to-application authentication using dynamic runtime behaviors. [Online]. Available: <https://arxiv.org/abs/2211.13195>
25. Jones MB, Nadalin A, Campbell B, Bradley J, Mortimore C (2020 Jan) OAuth 2.0 token exchange. Rfc 8693. [Online]. Available: <https://www.rfc-editor.org/info/rfc8693>
26. Amazon Web Services (2025) Simplify access to external services using AWS IAM outbound identity federation. [Online]. Available: <https://aws.amazon.com/blogs/aws/simplify-access-to-external-services-using-aws-iam-outbound-identity-federation/>. Last accessed 30 Mar 2026
27. Sakimura N, Bradley J, Jones MB, de Medeiros B, Mortimore C (2014) OpenID connect core 1.0. [Online]. Available: https://openid.net/specs/openid-connect-core-1_0.html. Last accessed 30 Mar 2026
28. Aws signature version 4." [Online]. Available: <https://docs.aws.amazon.com/AWS/latest/API/sigv4-auth-using-authorization-header.html>
29. Zero GP (2020) Hashicorp vault auth issues. [Online]. Available: <https://googleprojectzero.blogspot.com/2020/10/enter-the-vault-auth-issues-hashicorp-vault.html>. Last accessed 15 Oct 2023
30. Jones MB, Bradley J, Sakimura N (2015 May) JSON web token (JWT). RFC 7519. [Online]. Available: <https://www.rfc-editor.org/info/rfc7519>

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.